```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>AI Path-Finding Simulation</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: linear-
gradient(135deg, #667eea 0%, #764ba2 100%); padding: 20px; }
        .container { max-width: 1400px; margin: 0 auto; background:
white; border-radius: 15px; padding: 30px; box-shadow: 0 20px 60px
rgba(0,0,0,0.3); }
        h1 { text-align: center; color: #2d3748; margin-bottom:
10px; }
        .subtitle { text-align: center; color: #718096; margin-
bottom: 30px; }
        .guide { background: #ebf8ff; border-left: 4px solid
#3182ce; padding: 20px; margin-bottom: 30px; border-radius: 8px;
position: relative; }
        .guide h3 { color: #2c5282; margin-bottom: 15px; }
        .guide ol { margin-left: 20px; line-height: 1.8; }
        .guide-close { position: absolute; top: 20px; right: 20px;
background: none; border: none; font-size: 1.5em; cursor: pointer;
color: #3182ce; }
        .main-content { display: grid; grid-template-columns: 2fr
1fr; gap: 30px; margin-bottom: 30px; }
        .section { background: #f7fafc; padding: 25px; border-
radius: 10px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }
        .grid-header { display: flex; justify-content: space-
between; align-items: center; margin-bottom: 20px; flex-wrap: wrap;
gap: 10px; }
        .edit-buttons { display: flex; gap: 10px; flex-wrap: wrap; }
        .btn { padding: 10px 20px; border: none; border-radius: 8px;
cursor: pointer; font-weight: 600; transition: all 0.3s; }
        .btn:hover { transform: translateY(-2px); }
        .btn-start { background: #48bb78; color: white; }
        .btn-start.active { background: #2f855a; }
        .btn-goal { background: #f56565; color: white; }
        .btn-goal.active { background: #c53030; }
        .btn-obstacle { background: #4a5568; color: white; }
        .btn-obstacle.active { background: #2d3748; }
        .btn-run { background: #4299e1; color: white; font-size:
1.1em; }
        .btn-reset { background: #718096; color: white; }
        .btn-view { background: white; color: #2d3748; border: 2px
solid #e2e8f0; }
        .btn-view.active { background: #4299e1; color: white; }
        .btn-analysis { background: #48bb78; color: white; }
        .edit-hint { background: #fefcbf; border: 1px solid #f6e05e;
padding: 12px; border-radius: 6px; margin-bottom: 15px; color:
#744210; }
        .grid-container { display: inline-block; border: 3px solid
```

```
#4a5568; border-radius: 8px; overflow: hidden; margin-bottom:
20px; }
        .grid-row { display: flex; }
        .grid-cell { width: 50px; height: 50px; border: 1px solid
#cbd5e0; transition: all 0.2s; }
        .grid-cell.clickable { cursor: pointer; }
        .grid-cell.clickable:hover { opacity: 0.7; }
        .cell-start { background: #48bb78; }
        .cell-goal { background: #f56565; }
        .cell-obstacle { background: #2d3748; }
        .cell-bfs { background: #63b3ed; }
        .cell-dfs { background: #b794f4; }
        .cell-astar { background: #f6e05e; }
        .cell-empty { background: #edf2f7; }
        .cell-highlight { box-shadow: inset 0 0 0 4px #3182ce; }
        .legend { display: grid; grid-template-columns: 1fr 1fr;
gap: 20px; margin-top: 20px; }
        .legend-item { display: flex; align-items: center; gap:
10px; margin-bottom: 8px; }
        .legend-box { width: 30px; height: 30px; border: 1px solid
#cbd5e0; border-radius: 4px; }
        .controls-section { display: flex; flex-direction: column;
gap: 20px; }
        .control-buttons { display: flex; flex-direction: column;
gap: 10px; }
        .stats-table { background: #f7fafc; padding: 25px; border-
radius: 10px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); margin-bottom:
30px; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 12px; text-align: left; }
        th:not(:first-child), td:not(:first-child) { text-align:
right; }
        thead tr { border-bottom: 2px solid #4a5568; }
        .row-bfs { background: #bee3f8; }
        .row-dfs { background: #e9d8fd; }
        .row-astar { background: #fefcbf; }
        .analysis { background: #f7fafc; padding: 30px; border-
radius: 10px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }
        .algo-card { border-left: 4px solid; padding: 20px; margin-
bottom: 20px; background: white; border-radius: 8px; }
        .algo-card.bfs { border-color: #4299e1; }
        .algo-card.dfs { border-color: #9f7aea; }
        .algo-card.astar { border-color: #ecc94b; }
        .algo-card h3 { margin-bottom: 15px; }
        .algo-card ul { margin-left: 20px; line-height: 1.8; }
        .hidden { display: none; }
        @media (max-width: 1200px) {
            .main-content { grid-template-columns: 1fr; }
            .grid-cell { width: 40px; height: 40px; }
        }
    </style>
</head>
<body>
    <div class="container">
```

```
        <h1>🤖 AI Path-Finding Simulation</h1>
        <p class="subtitle">Comparing BFS, DFS, and A* Algorithms</
p>

        <div id="guide" class="guide">
            <button class="guide-close" onclick="toggleGuide()">×</
button>
            <h3>📋 Quick Start Guide</h3>
            <ol>
                <li><strong>Set Start:</strong> Click "Set Start",
then click any empty cell</li>
                <li><strong>Set Goal:</strong> Click "Set Goal",
then click any empty cell</li>
                <li><strong>Edit Obstacles:</strong> Click "Edit
Obstacles", then click cells to toggle</li>
                <li><strong>Run:</strong> Click "Run All Algorithms"
to see paths</li>
                <li><strong>Compare:</strong> View performance table
below</li>
                <li><strong>Analyze:</strong> Click "Show Analysis"
for detailed comparison</li>
            </ol>
        </div>

        <div class="main-content">
            <div class="section">
                <div class="grid-header">
                    <h2>Grid (10x10)</h2>
                    <div class="edit-buttons">
                        <button class="btn btn-start"
onclick="setEditMode('start')">🎯 Set Start</button>
                        <button class="btn btn-goal"
onclick="setEditMode('goal')">🏁 Set Goal</button>
                        <button class="btn btn-obstacle"
onclick="setEditMode('obstacle')">🧱 Edit Obstacles</button>
                    </div>
                </div>
                <div id="editHint" class="edit-hint hidden"></div>
                <div class="grid-container" id="gridContainer"></
div>
                <div class="control-buttons">
                    <button class="btn btn-run"
onclick="runAll()">▶️ Run All Algorithms</button>
                    <button class="btn btn-reset"
onclick="reset()">🔄 Reset Grid</button>
                </div>
                <div class="legend">
                    <div>
                        <h4>Legend:</h4>
                        <div class="legend-item"><div class="legend-
box cell-start"></div><span>Start</span></div>
                        <div class="legend-item"><div class="legend-
```

```
box cell-goal"></div><span>Goal</span></div>
                        <div class="legend-item"><div class="legend-
box cell-obstacle"></div><span>Obstacle</span></div>
                    </div>
                    <div>
                        <h4>Paths:</h4>
                        <div class="legend-item"><div class="legend-
box cell-bfs"></div><span>BFS</span></div>
                        <div class="legend-item"><div class="legend-
box cell-dfs"></div><span>DFS</span></div>
                        <div class="legend-item"><div class="legend-
box cell-astar"></div><span>A*</span></div>
                    </div>
                </div>
            </div>

            <div class="controls-section">
                <div class="section">
                    <h3>View Options</h3>
                    <div class="control-buttons">
                        <button class="btn btn-view active"
onclick="setView('all')">Show All</button>
                        <button class="btn btn-view"
onclick="setView('bfs')">BFS Only</button>
                        <button class="btn btn-view"
onclick="setView('dfs')">DFS Only</button>
                        <button class="btn btn-view"
onclick="setView('astar')">A* Only</button>
                    </div>
                </div>
                <div class="section">
                    <h3>Options</h3>
                    <div class="control-buttons">
                        <button class="btn btn-analysis"
onclick="toggleAnalysis()">📊 Show Analysis</button>
                        <button class="btn btn-view"
onclick="toggleGuide()">📖 Show Guide</button>
                    </div>
                </div>
            </div>
        </div>

        <div id="statsTable" class="stats-table hidden">
            <h2>Performance Comparison</h2>
            <table>
                <thead><tr><th>Algorithm</th><th>Path Length</
th><th>Nodes Explored</th><th>Max Memory</th><th>Time (ms)</th></
tr></thead>
                <tbody id="statsBody"></tbody>
            </table>
        </div>

        <div id="analysis" class="analysis hidden">
            <h2>Algorithm Analysis</h2>
```

```
            <div class="algo-card bfs">
                <h3>Breadth-First Search (BFS)</h3>
                <h4>Strengths:</h4>
                <ul>
                    <li>Guarantees shortest path in unweighted
graphs</li>
                    <li>Complete - always finds solution if exists</
li>
                    <li>Systematic level-by-level exploration</li>
                </ul>
                <h4>Weaknesses:</h4>
                <ul>
                    <li>High memory usage - stores all nodes at
current depth</li>
                    <li>Explores many unnecessary nodes</li>
                    <li>No heuristic guidance</li>
                </ul>
                <p><strong>Best Used:</strong> When shortest path is
critical and graph is small/medium sized.</p>
            </div>
            <div class="algo-card dfs">
                <h3>Depth-First Search (DFS)</h3>
                <h4>Strengths:</h4>
                <ul>
                    <li>Low memory - stores only current path</li>
                    <li>Fast for finding any solution</li>
                    <li>Efficient in maze-like environments</li>
                </ul>
                <h4>Weaknesses:</h4>
                <ul>
                    <li>No shortest path guarantee</li>
                    <li>Can explore long wrong paths</li>
                    <li>May find suboptimal solutions</li>
                </ul>
                <p><strong>Best Used:</strong> When any solution
acceptable and memory limited.</p>
            </div>
            <div class="algo-card astar">
                <h3>A* Search</h3>
                <h4>Strengths:</h4>
                <ul>
                    <li>Optimal with admissible heuristic</li>
                    <li>Fewer nodes explored than BFS</li>
                    <li>Best speed/optimality balance</li>
                    <li>Highly practical</li>
                </ul>
                <h4>Weaknesses:</h4>
                <ul>
                    <li>Requires good heuristic</li>
                    <li>More memory than DFS</li>
                    <li>More complex implementation</li>
                </ul>
                <p><strong>Best Used:</strong> When shortest path
needed with good heuristic (most real applications).</p>
```

```
            </div>
        </div>
    </div>

    <script>
        const SIZE = 10;
        let start = {x:1,y:1}, goal = {x:8,y:8}, obstacles = [],
bfsPath = [], dfsPath = [], astarPath = [];
        let editMode = null, view = 'all';

        function init() {
            obstacles = [{x:3,y:2},{x:3,y:3},{x:3,y:4},{x:3,y:5},
{x:6,y:4},{x:6,y:5},{x:6,y:6},{x:6,y:7},{x:2,y:7},{x:3,y:7},
{x:4,y:7}];
            render();
        }

        function render() {
            const c = document.getElementById('gridContainer');
            c.innerHTML = '';
            for(let y=0; y<SIZE; y++) {
                const r = document.createElement('div');
                r.className = 'grid-row';
                for(let x=0; x<SIZE; x++) {
                    const cell = document.createElement('div');
                    cell.className = 'grid-cell ' + getClass(x,y);
                    if(editMode) cell.classList.add('clickable');
                    cell.onclick = () => click(x,y);
                    r.appendChild(cell);
                }
                c.appendChild(r);
            }
        }

        function getClass(x,y) {
            if(x===start.x && y===start.y) return 'cell-start'+
(editMode==='start'?' cell-highlight':'');
            if(x===goal.x && y===goal.y) return 'cell-goal'+
(editMode==='goal'?' cell-highlight':'');
            if(obstacles.some(o=>o.x===x&&o.y===y)) return 'cell-
obstacle';
            if(view==='bfs' && bfsPath.some(p=>p.x===x&&p.y===y))
return 'cell-bfs';
            if(view==='dfs' && dfsPath.some(p=>p.x===x&&p.y===y))
return 'cell-dfs';
            if(view==='astar' &&
astarPath.some(p=>p.x===x&&p.y===y)) return 'cell-astar';
            if(view==='all') {
                if(bfsPath.some(p=>p.x===x&&p.y===y)) return 'cell-
bfs';
                if(dfsPath.some(p=>p.x===x&&p.y===y)) return 'cell-
dfs';
                if(astarPath.some(p=>p.x===x&&p.y===y)) return
'cell-astar';
```

```
            }
            return 'cell-empty';
        }

        function click(x,y) {
            if(!editMode) return;
            clear();
            if(editMode==='start' && (x!==goal.x||y!==goal.y) && !
obstacles.some(o=>o.x===x&&o.y===y)) {
                start = {x,y}; setEditMode(null);
            } else if(editMode==='goal' && (x!==start.x||y!
==start.y) && !obstacles.some(o=>o.x===x&&o.y===y)) {
                goal = {x,y}; setEditMode(null);
            } else if(editMode==='obstacle' && (x!==start.x||y!
==start.y) && (x!==goal.x||y!==goal.y)) {
                const i = obstacles.findIndex(o=>o.x===x&&o.y===y);
                i!==-1 ? obstacles.splice(i,1) :
obstacles.push({x,y});
            }
            render();
        }

        function setEditMode(m) {
            editMode = editMode===m ? null : m;
            const h = document.getElementById('editHint');
            document.querySelectorAll('.btn-start, .btn-goal, .btn-
obstacle').forEach(b=>b.classList.remove('active'));
            if(editMode==='start') { h.textContent='👆 Click empty
cell for START'; h.classList.remove('hidden');
document.querySelector('.btn-start').classList.add('active'); }
            else if(editMode==='goal') { h.textContent='👆 Click
empty cell for GOAL'; h.classList.remove('hidden');
document.querySelector('.btn-goal').classList.add('active'); }
            else if(editMode==='obstacle') { h.textContent='👆
Click to ADD/REMOVE obstacles'; h.classList.remove('hidden');
document.querySelector('.btn-obstacle').classList.add('active'); }
            else h.classList.add('hidden');
            render();
        }

        function clear() {
            bfsPath = []; dfsPath = []; astarPath = [];

document.getElementById('statsTable').classList.add('hidden');
        }

        function reset() {
            start = {x:1,y:1}; goal = {x:8,y:8};
            init(); clear(); setEditMode(null);
        }

        function neighbors(n) {
            const d = [{x:0,y:-1},{x:1,y:0},{x:0,y:1},{x:-1,y:0}], r
```

```
= [];
            for(const {x:dx,y:dy} of d) {
                const nx=n.x+dx, ny=n.y+dy;
                if(nx>=0 && nx<SIZE && ny>=0 && ny<SIZE && !
obstacles.some(o=>o.x===nx&&o.y===ny)) r.push({x:nx,y:ny});
            }
            return r;
        }

        function bfs() {
            const t = performance.now(), q = [{...start,path:
[start]}], v = new Set([`${start.x},${start.y}`]);
            let n = 0, m = 1;
            while(q.length) {
                m = Math.max(m, q.length);
                const c = q.shift(); n++;
                if(c.x===goal.x && c.y===goal.y) return
{path:c.path, stats:{pathLength:c.path.length, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
                for(const nb of neighbors(c)) {
                    const k = `${nb.x},${nb.y}`;
                    if(!v.has(k)) { v.add(k); q.push({...nb,path:
[...c.path,nb]}); }
                }
            }
            return {path:[], stats:{pathLength:0, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
        }

        function dfs() {
            const t = performance.now(), s = [{...start,path:
[start]}], v = new Set();
            let n = 0, m = 1;
            while(s.length) {
                m = Math.max(m, s.length);
                const c = s.pop(), k = `${c.x},${c.y}`;
                if(v.has(k)) continue;
                v.add(k); n++;
                if(c.x===goal.x && c.y===goal.y) return
{path:c.path, stats:{pathLength:c.path.length, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
                for(const nb of neighbors(c)) {
                    const nk = `${nb.x},${nb.y}`;
                    if(!v.has(nk)) s.push({...nb,path:
[...c.path,nb]});
                }
            }
            return {path:[], stats:{pathLength:0, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
        }

        function astar() {
            const h = n => Math.abs(n.x-goal.x)+Math.abs(n.y-
goal.y);
```

```
            const t = performance.now(), o = [{...start,path:
[start],g:0,f:h(start)}], v = new Set();
            let n = 0, m = 1;
            while(o.length) {
                m = Math.max(m, o.length);
                o.sort((a,b)=>a.f-b.f);
                const c = o.shift(), k = `${c.x},${c.y}`;
                if(v.has(k)) continue;
                v.add(k); n++;
                if(c.x===goal.x && c.y===goal.y) return
{path:c.path, stats:{pathLength:c.path.length, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
                for(const nb of neighbors(c)) {
                    const nk = `${nb.x},${nb.y}`;
                    if(!v.has(nk)) {
                        const g = c.g+1, f = g+h(nb);
                        o.push({...nb,path:[...c.path,nb],g,f});
                    }
                }
            }
            return {path:[], stats:{pathLength:0, nodesExplored:n,
maxMemory:m, time:(performance.now()-t).toFixed(4)}};
        }

        function runAll() {
            const r1 = bfs(), r2 = dfs(), r3 = astar();
            bfsPath = r1.path; dfsPath = r2.path; astarPath =
r3.path;
            const b = document.getElementById('statsBody');
            b.innerHTML = `
                <tr class="row-bfs"><td>BFS</td><td>$
{r1.stats.pathLength}</td><td>${r1.stats.nodesExplored}</td><td>$
{r1.stats.maxMemory}</td><td>${r1.stats.time}</td></tr>
                <tr class="row-dfs"><td>DFS</td><td>$
{r2.stats.pathLength}</td><td>${r2.stats.nodesExplored}</td><td>$
{r2.stats.maxMemory}</td><td>${r2.stats.time}</td></tr>
                <tr class="row-astar"><td>A*</td><td>$
{r3.stats.pathLength}</td><td>${r3.stats.nodesExplored}</td><td>$
{r3.stats.maxMemory}</td><td>${r3.stats.time}</td></tr>
            `;

document.getElementById('statsTable').classList.remove('hidden');
            render();
        }

        function setView(v) {
            view = v;
            document.querySelectorAll('.btn-
view').forEach(b=>b.classList.remove('active'));
            event.target.classList.add('active');
            render();
        }

        function toggleGuide() {
```

```
document.getElementById('guide').classList.toggle('hidden');
        }

        function toggleAnalysis() {

document.getElementById('analysis').classList.toggle('hidden');
        }

        init();
    </script>
</body>
</html>
```